MyCodeSchool

Exerciții — șiruri de caractere

Python · proiectul `py-siruri` · Luca

Ce ai voie: tot ce ai folosit la cifre · indexare `text[0]`, `text[-1]` · felii `text[:3]`, `text[::-1]` · `len`, `in` · `for litera in text` · `.lower()`, `.upper()`, `.split()`, `.replace()`.
Încă NU folosim: liste construite de tine, `print` în funcții.
Textul gol `""` e cazul tău cu 0: unde apare în tabel, funcția trebuie să-l ducă la capăt fără eroare.

Un șir e o înșiruire de litere, fiecare cu poziția ei

poziție	0	1	2	3	4	5
literă	P	y	t	h	o	n
de la coadă	−6	−5	−4	−3	−2	−1

scrii	primești	ce face
`text[0]`	`"P"`	prima literă — pozițiile pornesc de la 0
`text[-1]`	`"n"`	ultima literă
`text[1:4]`	`"yth"`	felie: de la poziția 1 până **înainte** de 4
`text[:3]`	`"Pyt"`	felie de la început
`text[::-1]`	`"nohtyP"`	tot șirul, de la coadă
`len(text)`	6	câte litere are
`"y" in text`	`True`	apare sau nu
`"Ana".lower()`	`"ana"`	totul cu litere mici; `.upper()` = mari
`"ana are".split()`	`["ana", "are"]`	cuvintele, despărțite la spații
`"a b".replace(" ", "")`	`"ab"`	înlocuiește peste tot

Parcurgerea literă cu literă — primul tău `for` în Python. Nu mai ai contor de poziție: `for` îți dă direct fiecare literă, pe rând.

```
for litera in "Luca":
    print(litera)
```

Atenție: un șir nu se poate modifica. `text.lower()` nu schimbă `text`, ci **întoarce** un șir nou — dacă nu-l pui într-o variabilă, s-a pierdut.

Cum lucrezi: `siruri.py` calculează, `main.py` afișează

Ca la cifre: fiecare exercițiu e o funcție în `siruri.py` care întoarce cu `return`. `main.py` pornește așa:

```
from siruri import prima_litera


def main():
    print(prima_litera("Python"))
    print(prima_litera("a"))


if __name__ == "__main__":
    main()
```

Ieșire așteptată:

```
P
a
```

La fiecare funcție nouă: o adaugi în `import` și pui câte un `print` pentru fiecare rând din tabelul ei. `print` pe un șir nu arată ghilimelele — în tabel le vezi ca să știi că e text.

La clasă

Prima și ultima literă 1

```
def prima_litera(text: str) -> str:
def ultima_litera(text: str) -> str:
```

apel	întoarce	apel	întoarce
`prima_litera("Python")`	`"P"`	`ultima_litera("Python")`	`"n"`
`prima_litera("a")`	`"a"`	`ultima_litera("a")`	`"a"`

Câte o linie fiecare. Aici textul are mereu cel puțin o literă: pe `""` nu există poziția 0 și programul se oprește cu eroare — o rezolvăm la lecția despre excepții.

Primele trei litere 2

```
def primele_trei(text: str) -> str:
```

apel	întoarce
`primele_trei("Python")`	`"Pyt"`
`primele_trei("ab")`	`"ab"`
`primele_trei("")`	`""`

O felie. Uită-te la ultimele două rânduri: felia nu dă eroare dacă textul e mai scurt — ia cât există.

Textul inversat 3

```
def inversat(text: str) -> str:
```

apel	întoarce
`inversat("Python")`	`"nohtyP"`
`inversat("ana")`	`"ana"`
`inversat("")`	`""`

La cifre, `oglindit` a cerut o buclă întreagă. Aici e o felie. Compară-le.

Mai grele

Câte vocale 4

```
def numar_vocale(text: str) -> int:
```

apel	întoarce
`numar_vocale("Luca Todor")`	`4`
`numar_vocale("xyz")`	`0`
`numar_vocale("ÎNVĂȚ")`	`2`
`numar_vocale("")`	`0`

Contor + `for litera in text`. Ca să numeri și vocalele mari, parcurge `text.lower()`. Verificarea „e vocală” e un singur `in`: `litera in "aeiouăâî"`.

Câte cuvinte 5

```
def numar_cuvinte(text: str) -> int:
```

apel	întoarce
`numar_cuvinte("ana are mere")`	`3`
`numar_cuvinte(" doua spatii ")`	`2`
`numar_cuvinte("")`	`0`

Dacă numeri spațiile și aduni 1, rândul doi îți dă 7, iar ultimul 1. `.split()` fără nimic în paranteze sare peste oricâte spații.

Inițialele 6

```
def initiale(nume_complet: str) -> str:
```

apel	întoarce
initiale("Luca Todor")	"LT"
initiale("ana maria pop")	"AMP"
initiale("")	""

Acumulator, dar de text: pornești de la `rezultat = ""` și adaugi cu `+=`. Parcurgi cuvintele, nu literele: `for cuvant in nume_complet.split():`.

Cuvântul cel mai lung 7

```
def cuvantul_cel_mai_lung(text: str) -> str:
```

apel	întoarce
cuvantul_cel_mai_lung("ana are mere")	"mere"
cuvantul_cel_mai_lung("un exercitiu bun")	"exercitiu"
cuvantul_cel_mai_lung("ana are")	"ana"
cuvantul_cel_mai_lung("")	""

*Ca `cifra_maxima`, dar compari `len`-urile. Rândul trei: la egalitate rămâne **primul** — deci `>`, nu `>=`.*

Provocări

Fără indicii.

Palindrom, pe text 8

```
def este_palindrom(text: str) -> bool:
```

Nu contează spațiile și nici literele mari.

apel	întoarce
este_palindrom("Ele fac cafele")	True
este_palindrom("Ana")	True
este_palindrom("Python")	False
este_palindrom("")	True

Suma cifrelor, a doua oară 9

```
def suma_cifrelor(n: int) -> int:
```

Aceeași funcție ca în `py-initiere`, dar **fără `%` și fără `//`**: transformi numărul în text cu `str(n)`, iar fiecare literă înapoi în număr cu `int(...)`.

apel	întoarce
suma_cifrelor(4738)	22
suma_cifrelor(1531)	10
suma_cifrelor(0)	0

La final scrie în `main.py`, ca și comentariu, care din cele două variante ți se pare mai clară și de ce.

Înainte să faci push, treci fiecare funcție prin astea:

1. Am pus câte un `print` pentru **fiecare rând** din tabel, inclusiv cel cu `""`? Ieșirea e identică cu coloana „întoarce”?
2. Apare `None` undeva? Atunci o funcție afișează în loc să întoarcă.
3. Am apelat `.lower()` sau `.replace()` fără să păstrez rezultatul?
4. Am scris de două ori aceeași buclă? Dacă o funcție o are deja, o chem.